

Пицца

(Время: 1 сек. Память: 16 Мб Сложность: 20%)

Пицца – любимое лакомство Васи, он постоянно покупает и с удовольствием употребляет различные сорта этого великолепного блюда. Однажды, в очередной раз, разрезая крупную пиццу на несколько частей, Вася задумался: на какое максимальное количество частей можно разрезать пиццу за N прямых разрезов?

Помогите Васе решить эту задачу, определив максимальное число не обязательно равных кусков, которые может получить Вася, разрезая пиццу таким образом.

Входные данные

Входной файл INPUT.TXT содержит натуральное число N – число прямых разрезов пиццы ($N \leq 1000$).

Выходные данные

В выходной файл OUTPUT.TXT выведите ответ на задачу.

Примеры

№	INPUT.TXT	OUTPUT.TXT
1	2	4
2	3	7

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main()
{
    int n;
    cin >> n;
    cout << n * (n + 1) / 2 + 1;
}
```

Гвоздики

(Время: 1 сек. Память: 16 Мб Сложность: 34%)

На прямой доске вбиты гвоздики. Любые два гвоздика можно соединить ниточкой. Требуется соединить некоторые пары гвоздиков ниточками так, чтобы к каждому гвоздику была привязана хотя бы одна ниточка, а суммарная длина всех ниточек была минимальна.

Входные данные

В первой строке входного файла INPUT.TXT записано число N - количество гвоздиков ($2 \leq N \leq 100$). В следующей строке записано N чисел - координаты всех гвоздиков (неотрицательные целые числа, не превосходящие 10000).

Выходные данные

В выходной файл OUTPUT.TXT нужно вывести единственное число - минимальную суммарную длину всех ниточек.

Пример

№	INPUT.TXT	OUTPUT.TXT
1	 6 3 4 12 6 14 13	5 

```
#include <iostream>
#include <algorithm>

using namespace std;

int n;
int a[111];
int dp[111];

int main(){
    cin >> n;
    for(int i = 0; i < n; ++ i){
        cin >> a[i];
    }
    sort(a, a + n);
    dp[1] = a[1] - a[0];
    dp[2] = a[2] - a[0];

    for(int i = 3; i < n; ++ i){
        dp[i] = min(dp[i - 1], dp[i - 2]) + a[i] - a[i - 1];
    }
    cout << dp[n - 1];
    return 0;
}
```

Агент

(Время: 1 сек. Память: 16 Мб Сложность: 34%)

Агент Джеймс Бонд пошел на пенсию, но неутомимый характер требовал новых впечатлений. Поэтому Джеймс Бонд с удовольствием согласился провести мастер-класс в некоторых группах школы «Молодого агента». Тема одного из занятий – работа агента с напарником. В таком опасном деле, как разведка, важно иметь очень надёжного напарника, поэтому напарниками могут стать только агенты, которые максимально близки по возрасту (т.е. два агента не могут стать напарниками, если в группе существует третий агент, который старше одного и младше другого).

Задание Бонда состоит в том, чтобы агенты нашли друг другу напарников таким образом, чтобы у каждого агента был хотя бы один напарник (всего у агента может быть 2 напарника – один младше, и один старше него, но эти двое не считаются напарниками между собой). Очевидно, что группа из 4 и более агентов может поделиться несколькими способами.

После нескольких занятий Бонд узнал способности групп, обучающихся в школе «Молодого агента», и оценил риск раскрытия каждого агента в отдельности. Но специфика работы с напарником такова, что в паре риску подвергается только старший из двух агентов, поэтому группу надо распределить так, чтобы суммарный риск был минимален.

Входные данные

В первой строке входного файла INPUT.TXT находится одно целое число N – количество агентов в группе ($2 \leq N \leq 10000$). Во второй строке находятся N пар целых положительных чисел, разделённых пробелом. Первое число в паре – это возраст агента (в днях) из диапазона $[5000, 16000]$, второе – риск раскрытия агента, число в диапазоне $[1, 1000]$. Известно, что в любой группе все агенты разного возраста.

Выходные данные

В выходной файл OUTPUT.TXT выведите единственное число – минимальное значение суммарного риска раскрытия группы.

Примеры

№	INPUT.TXT	OUTPUT.TXT
1	3 6000 2 5500 3 5000 4	5
2	5 5005 1 5004 2 5003 3 5002 4 5001 5	7

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N;
    if (!(cin >> N)) return 0;
    vector<pair<int,int>> a(N); // {age, risk}
    for (int i = 0; i < N; ++i) cin >> a[i].first >> a[i].second;

    // 1) Yosh bo'yicha saralash (oshish tartibida)
    sort(a.begin(), a.end());

    // 2) Faqat risklar ketma-ketligini olib qo'yamiz (saralangan yoshga mos)
    vector<int> risk(N+1);
    for (int i = 1; i <= N; ++i) risk[i] = a[i-1].second;

    const long long INF = (1LL<<60);
    // dp[i][c]: 1..i agentlar ko'rib chiqilgan, c=0 -> i qoplanmagan, c=1 -> i
    // qoplan
    vector<array<long long,2>> dp(N+1, {INF, INF});
    dp[1][0] = 0; // hali hech qanday qirra tanlanmagan, 1-agent qoplanmagan

    for (int i = 1; i <= N-1; ++i) {
        // 1) Qirrani olish (i, i+1) – xarajat risk[i+1]
        // Bu i va i+1 ni qoplaydi; keyingi holatda (i+1) qoplanagan bo'ladi
        dp[i+1][1] = min(dp[i+1][1], dp[i][0] + risk[i+1]);
        dp[i+1][1] = min(dp[i+1][1], dp[i][1] + risk[i+1]);
    }
```

```

// 2) Qirrani olmaslik – faqat i allaqachon qoplangan bo'lsa ruxsat
dp[i+1][0] = min(dp[i+1][0], dp[i][1]);
}

// Oxirgi agent ham qoplangan bo'lishi kerak
cout << dp[N][1] << "\n";
return 0;
}

```

Masala modeliga tushuncha

- Har bir agentning yoshi hamma uchun noyob. Agar ikkita agent *partner* bo'lsa, ular orasida hech qanday uchinchi agentning yoshi bo'lmasligi kerak.
→ Demak, agar agentlarni yosh bo'yicha oshish tartibida joylashtirsak, faqat **qo'shni** agentlar (i va $i+1$) bir-biri bilan partner bo'lishi mumkin.
- Har bir agent kamida bitta partnerga ega bo'lishi kerak. Bitta agent maksimum ikkita partnerga ega bo'lishi mumkin (bir kichikroq, bir kattaroq) — bu yo'qotmaydi, chunki yo'l (path) grafigida max daraja 2.
- Juftlik tanlanganda ($i, i+1$) juftligi faqat kattaroq agent (ya'ni $i+1$) riskini keltirib chiqaradi. Demak, har bir tanlangan qirra (edge) uchun xarajat = `risk[i+1]`.

Shunday qilib, vazifa: yosh bo'yicha saralangan yo'l (path) ustida qirralar to'plamini tanlash (edge cover) — har bir vertex kamida bitta tanlangan qirraga ulanadi — va umumiy xarajat (tanlangan qirralar uchun) minimal bo'lsin.

Nima uchun faqat qo'shnilar?

Original shart: "ikki agent naparniki bo'la olmaydi, agar guruhda ularning yoshlariga o'rtada bo'lgan uchinchi agent mavjud bo'lsa." Bu aniq qo'yadi: faqat yosh tartibida yonma-yon joylashgan agentlar o'zaro partner bo'lishi mumkin. Shuning uchun masala **bir o'lchovli yo'l (path)** bo'yicha minimal og'irlikli edge-cover ga aylanadi.

DP model (koddagi ta'rif)

Aniqlik uchun indekslashni $1..N$ deb olaylik, agentlar yosh bo'yicha **oshish** tartibida saralangan va `risk[i]` — i -chi (saralangan) agentning riski.

Ta'rif:

`dp[i][c]` — birinchi i agentlarni ko'rib chiqqanimizda erishish mumkin bo'lgan minimal umumiy xarajat, bunda:

- `c = 1` — i -chi agent **qoplangan** (ya'ni $1..i$ dagi har bir agent kamida bitta partnerga ega);
- `c = 0` — $1..i-1$ agentlar to'liq qoplangan, lekin i -chi agent **hozircha qoplanmagan** (ya'ni u uchun hali $i-1$ bilan qirra olinmagan).

Boshlanish: $dp[1][0] = 0$ (1-chi agentni ilk holatda qoplamay boshlaymiz; hali hech qirra tanlanmagan).

Boshqa boshlang'ichlar = +INF.

Endi i dan $i+1$ ga o'tish qoidalarini (koddagi ikki ustunli o'tish):

1. Agar $(i, i+1)$ qirrasini olsak:

- Bu qirra xarajatiga $risk[i+1]$ to'lanadi (kattaroq agent = $i+1$ riskiga ta'sirlanadi).
 - Bu holatda $i+1$ qoplanadi, i esa qoplanishining oldingi holatidan qat'i nazar (u allaqachon qoplangan yoki yo'q) mumkin.
- Shunday qilib:

lua

 Копировать код

```
dp[i+1][1] = min(dp[i+1][1], dp[i][0] + risk[i+1])
dp[i+1][1] = min(dp[i+1][1], dp[i][1] + risk[i+1])
```



2. Agar $(i, i+1)$ qirrasini OLMASAK:

- Bu holatda $i+1$ qoplanmagan bo'lib qoladi — lekin natijada hech bir vertex yolg'iz qolmasligi kerak. Shuning uchun $i+1$ ni qoplanmagan qila olishimiz uchun i ALLAQACHON qoplanishi kerak (aks holda i ham $i+1$ ham qoplanmagan bo'lib qoladi — man etilgan).
- Shu sababli:

lua

 Копировать код

```
dp[i+1][0] = min(dp[i+1][0], dp[i][1])
```

Oxir-oqibat oxirgi agent N ham qoplanishi kerak, ya'ni natija $dp[N][1]$.

Nima uchun bu to'g'ri? (induktiv dalil)

Induktiv argument:

- Baza: $i=1$ uchun $dp[1][0]=0$ optimal — hech qirra olinmaganda xarajat 0, va bu holat 1-chi agent hozircha qoplanmaganligini ifodalaydi.
- Induktiv qadam: faraz qilaylik, biz $dp[i][0]$ va $dp[i][1]$ ga optimallik qiymatlarni topdik (ya'ni birinchi i agent uchun eng yaxshi qiymatlar saqlangan). Endi $i+1$ -chi agentni ko'rib chiqayotganda ikkita mustaqil qarorni ko'rib chiqdik: $(i, i+1)$ qirrasini olish yoki olmaslik — va bu ikkala holat ham barcha mumkin tashkil etishlarni qamrab oladi:
 - Agar biz $(i, i+1)$ olsak, xarajatga $risk[i+1]$ qo'shiladi va $i+1$ qoplanadi; oldingi i holatini esa $dp[i][0]$ yoki $dp[i][1]$ dan kelib tanlaymiz (ikkovadan yaxshisini olamiz).
 - Agar qirra olinmasa, yagona ruxsat etilgan holat — i avvaldan qoplangan bo'lishi; boshqa holatda i ham $i+1$ ham qoplanmagan bo'lib qoladi (man etilgan).
- Shuning uchun har bir prefix uchun optimal variantlar saqlanadi va oxirida $dp[N][1]$ global optimalni beradi.

Bu DP chuqur matematik jihatdan *minimal og'irlikli edge cover* ni hisoblaydi va to'liq izchil.

Misol bilan tushuntirish (1-chi test)

Kiritish: 3 agent: (6000,2) (5500,3) (5000,4)

Yosh bo'yicha oshish: (5000,4), (5500,3), (6000,2) → risklar [4,3,2].

Qirralar va xarajatlar:

- (1,2) xarajat = risk[2] = 3
- (2,3) xarajat = risk[3] = 2

Edge-cover shartiga binoan har bir vertex kamida bitta qirraga ega bo'lishi kerak. Variantlar:

- Ikkala qirra ham olinadi: xarajat = 3+2 = 5 (hamma qoplanadi).
- Faqat (1,2) olinsa: 3 xarajat, lekin 3-chi vertex (index 3) qoplanmagan — **yoq**.
- Faqat (2,3) olinsa: 2 xarajat, lekin 1-chi vertex qoplanmagan — **yoq**.

Eng yaxshi — ikkala qirra, jami 5. DP ham xuddi shu natijani beradi (dp[3][1]=5).

Murakkablik va resurslar

- Saralash: $O(N \log N)$ (yosh bo'yicha sort).
- DP o'tish: $O(N)$ (har i uchun bir nechta minima yangilanishi).
- Umumiy vaqt: $O(N \log N)$.
- Xotira: $O(N)$ (risk massivi va dp).

Qo'shimcha izohlar / optimallashtirish

- Agar agentlar eski tartibda kelsa va allaqachon saralangan bo'lsa, saralashni olib tashlab $O(N)$ ga tushish mumkin.
- Masalani boshqa yo'llar bilan ham ko'rib chiqish mumkin (masalan, dinamik dasturlashni boshqa holat ta'riflari bilan), lekin berilgan $dp[i][0/1]$ eng sodda va tushunarli variantlardan biridir.

A. DP jadali (qadam-baqadam)

Kodda ishlatilgan ta'rif: agentlar yosh bo'yicha saralangan; $risk[i]$ — i -chi agentning riski ($1..N$).

$dp[i][0]$ — birinchi i agentlar ichida $1..i-1$ to'liq qoplangan, lekin i -chi agent **hozircha qoplanmagan**.

$dp[i][1]$ — birinchi i agentlar ham **barcha** qoplangan (ya'ni i -chi ham qoplanadi).

Boshlanish:

- $dp[1][0] = 0$ (1-chi agent hali qoplanmagan).
- $dp[1][1] = +INF$ (1 agentni qoplanib tugatish uchun hozircha hech qanday qirra yo'q; INF bilan beradi).

O'tish qoidalarini ($i \rightarrow i+1$):

1. Agar qirra ($i, i+1$) olinadigan bo'lsa, xarajatga $risk[i+1]$ qo'shiladi va $i+1$ qoplanadi:
 - $dp[i+1][1] = \min(dp[i+1][1], dp[i][0] + risk[i+1])$
 - $dp[i+1][1] = \min(dp[i+1][1], dp[i][1] + risk[i+1])$
2. Agar qirra olinmasa, $i+1$ qoplanmagan qoladi, bu faqat i allaqachon qoplangan bo'lganda mumkin:
 - $dp[i+1][0] = \min(dp[i+1][0], dp[i][1])$

Natija: $dp[N][1]$ — javob (oxirgi agent ham qoplangan bo'lishi shart).

Konkrekt misol: 3 agent (kod misolidan 1-chi test)

Agentlar: $(6000, 2)$, $(5500, 3)$, $(5000, 4)$ → yosh bo'yicha oshish: $(5000, 4)$, $(5500, 3)$, $(6000, 2)$

Demak $risk = [4, 3, 2]$ indekslash $1..3$ bo'yicha.

Boshlang'ich:

- $dp[1][0] = 0$
- $dp[1][1] = INF$

Endi $i=1 \rightarrow i+1=2$ uchun:

- O'tish (olmasak): $dp[2][0] = \min(INF, dp[1][1]) = INF$ ($dp[1][1]$ INF)
- O'tish (olsak (1,2)): $dp[2][1] = \min(INF, dp[1][0] + risk[2]) = 0 + 3 = 3$
(shuningdek $dp[1][1] + risk[2] = INF + 3 = INF$, qaramaydi)

Jadval hozir:

- $dp[2][0] = INF$
- $dp[2][1] = 3$

Endi $i=2 \rightarrow i+1=3$ uchun:

- O'tish (olsak (2,3)):
 $dp[3][1] = \min(INF, dp[2][0] + risk[3]) = INF + 2 = INF$ (no effect)
 $dp[3][1] = \min(current, dp[2][1] + risk[3]) = 3 + 2 = 5$
- O'tish (olmasak): $dp[3][0] = \min(INF, dp[2][1]) = 3$

Jadval oxirida:

- $dp[3][0] = 3$ (bu holatda oxirgi agent qoplanmagan — yaroqsiz yakun uchun)
- $dp[3][1] = 5$ (barcha agentlar qoplangan)

Chunki oxirgi agent qoplanishi kerak, javob $dp[3][1] = 5$. (Bu mos keladi dastur chiqishiga.)

Kengaytirilgan qadam-jadval (umumiy ko'rinish)

Quyidagi jadval ko'rsatadi har qadamda qanday qiymatlar yangilanadi (simvolik):

$i=1$ (boshlang'ich)

- $dp[1][0] = 0$
- $dp[1][1] = INF$

$i \rightarrow i+1$ umumiy yangilanish:

- $dp[i+1][1] := \min(\text{old } dp[i+1][1], dp[i][0] + \text{risk}[i+1], dp[i][1] + \text{risk}[i+1])$
- $dp[i+1][0] := \min(\text{old } dp[i+1][0], dp[i][1])$

Bu jadvalni N bo'yicha to'liq ko'rish uchun shu formulalarni qadamma-qadam ishlatasiz — har qadamda $dp[*][0/1]$ ning ikki qiymati yangilanadi.

B. Rasmiy dalil (induksiya) — DP formulasi to'g'riligini isbotlash

Masala qayta yozilishi

Yosh bo'yicha saralangan N vertexli yo'l (path) mavjud: vertexlar $1..N$ va imkoniy qirralar faqat $(i, i+1)$. Har bir olgan qirra $(i, i+1)$ uchun xarajat = $\text{risk}[i+1]$. Bizga kerak: har bir vertex kamida bitta olgan qirranga ulanadigan qirralar to'plamini tanlab, jami xarajatni minimallashtirish. Bu — minimal og'irlikli edge-cover muammosi yo'l grafigida.

DP ta'rifining to'g'riligini isbotlash

Ta'rif: $dp[i][0]$ va $dp[i][1]$ saytida berilgan ma'nolarni qaytadan yozamiz:

- $dp[i][1]$ = birinchi i vertexlar uchun minimal jami xarajat, bunda **hamma** vertexlar $(1..i)$ qoplangan.
- $dp[i][0]$ = birinchi i vertexlar ichida $1..i-1$ qoplangan, lekin i **qoplanmagan** holat uchun minimal xarajat (agar bunday holatlar mavjud bo'lmasa, qiymat = $+INF$).

Biz quyidagi induktiv invarianti tasdiqlaymiz: DP o'tishlaridan keyin $dp[i][0]$ va $dp[i][1]$ haqiqatan ham yuqoridagi ma'nolarni ifodalaydi (ya'ni ular optimal qiymatlar).

Baza ($i = 1$):

- Agar $i=1$ bo'lsa, $dp[1][0] = 0$ — to'g'ri: hech qanday qirra tanlanmagan, 1-chi vertex qoplanmagan va xarajat 0.
- $dp[1][1] = +INF$ — to'g'ri: 1 vertexni qoplash uchun (yagona qirra mavjud emas) imkoni yo'q (INF bilan ifodalanadi).

Induktiv qadam: faraz qilaylik invarianti i uchun to'g'ri. Ko'rib chiqamiz $i+1$ uchun.

Har qanday optimal edge-cover konfiguratsiyasini $(1..i+1)$ ning barcha vertexlarni qoplagan eng kam xarajatli to'plam deb olaylik. Ikki mumkin holat bor:

1. $(i, i+1)$ qirrasi tanlangan.

Unda $i+1$ qoplanadi. Qolgan xarajat — $risk[i+1]$ + minimal xarajat, bu $1..i$ uchun qaysi holatdan kelganiga bog'liq:

- Agar i gacha bo'lgan konfiguratsiyada i qoplanmagan bo'lgan holda bu qirra i ni ham qoplaydi; shunday qilib qolgan minimal xarajat $dp[i][0]$. Natijada umumiy qiymat $dp[i][0] + risk[i+1]$.
- Agar i allaqachon qoplangan bo'lsa, qolgan minimal xarajat $dp[i][1]$. Natija $dp[i][1] + risk[i+1]$.

Optimal konfiguratsiyaning umumiy qiymati shu ikki qiymatdan kichigi — bu bizning yangilanish qoidasi $dp[i+1][1] = \min(dp[i+1][1], dp[i][0] + risk[i+1], dp[i][1] + risk[i+1])$ bilan mos keladi.

2. $(i, i+1)$ qirrasi tanlanmagan.

Bu holatda $i+1$ ga uni qoplash uchun faqat $(i+1, i+2)$ kabi keyingi qirra yordam beradi (ya'ni tegishli bo'lsa), lekin shu nuqtada agar i ham qoplanmagan bo'lsa, i ham $i+1$ ham qoplanmagan bo'lib qoladi — bu oxirgi konfiguratsiyada qabul qilinmaydi (0 dan kam bo'lgan vertex bo'lmasligi kerak). Shu sabab, $(i, i+1)$ olinmaydigan optimal konfiguratsiya faqat i avvaldan **qoplangan** bo'lsa mumkin. Agar shunday bo'lsa, $dp[i+1][0]$ uchun qolgan xarajat teng $dp[i][1]$. Bu ham koddagi $dp[i+1][0] = \min(dp[i+1][0], dp[i][1])$ bilan ifodalanadi.

Bu ikki holat barcha mumkin bo'lgan optimal konfiguratsiyalarni qamrab oladi — shuning uchun induktiv tarzda $dp[i+1][0]$ va $dp[i+1][1]$ haqiqatan ham optimal qiymatlarni ifodalaydi.

Xulosa: Induksiya orqali DP formulalari butun domen ($i=1..N$) uchun to'g'ri ekanligi isbotlandi. Natija $dp[N][1]$ minimal umumiy xarajatga teng.

Qo'shimcha eslatmalar

- Bu DP N bo'yicha chiziqli ($O(N)$) va saralash $O(N \log N)$ talab qiladi (agar agentlar avvaldan saralanmagan bo'lsa).
- INF ga ehtiyot bo'ling — C++ da unda `long long` ostida katta qiymat (masalan $1e18$) ishlatish tavsiya etiladi.
- $dp[i][0]$ hamma i uchun haqiqiy bo'lmasligi mumkin — agar `INF` qolsa, demak shu prefix uchun i qoplanmagan holatga erishish mumkin emas.

Без двух нулей подряд

(Время: 1 сек. Память: 16 Мб Сложность: 37%)

Требуется вычислить количество N -значных чисел в системе счисления с основанием K , таких что их запись не содержит двух подряд идущих нулей.

Входные данные

Во входном файле INPUT.TXT записаны два натуральных числа N и K в десятичной системе счисления ($2 \leq K \leq 10$; $2 \leq N$; $4 \leq N+K \leq 18$).

Выходные данные

В выходной файл OUTPUT.TXT необходимо вывести целое число в десятичной записи – ответ на задачу.

Примеры

№	INPUT.TXT	OUTPUT.TXT
1	2 10	90
2	4 2	5
3	6 3	328

```
#include <bits/stdc++.h>

using namespace std;

typedef long long LL;

int cnt[20];

int main()
{
    int n, base;
    cin >> n >> base;
    cnt[1] = base - 1;
    cnt[2] = (base - 1) * base;
    for(int i = 3; i <= n; i++){
        cnt[i] = (base - 1) * (cnt[i - 1] + cnt[i - 2]);
    }
    cout << cnt[n];
}
```

Условие (кратко)

Найти число N -значных чисел в системе с основанием K (цифры $0, 1, \dots, K - 1$), такие что запись:

- не содержит двух подряд идущих нулей, и
- первая цифра не равна нулю (иначе это не N -значное число).

Ограничения: $2 \leq K \leq 10$, $2 \leq N$, $4 \leq N + K \leq 18$.

Идея решения (DP с учётом последней цифры)

Будем считать последовательности слева направо. Введём два числа для длины n :

- x_n — количество допустимых n -значных записей, заканчивающихся ненулевой цифрой;
- y_n — количество допустимых n -значных записей, заканчивающихся нулём.

Тогда искомое число для длины n равно

$$\text{cnt}_n = x_n + y_n.$$

Теперь найдём рекуррентные соотношения.

База

- Для $n = 1$: первая (и единственная) цифра не может быть нулём, значит

$$x_1 = K - 1, \quad y_1 = 0,$$

$$\text{и } \text{cnt}_1 = K - 1.$$

Переходы

Рассмотрим добавление последней (n -й) цифры к допустимой последовательности длины $n - 1$.

1. Чтобы получить последовательность длины n , **заканчивающуюся ненулём**:

- последняя цифра выбирается из $K - 1$ вариантов (1..K-1),
- предыдущее (первые $n - 1$) место может быть любым допустимым окончанием (ноль или не ноль),
значит

$$x_n = (K - 1) \cdot (x_{n-1} + y_{n-1}).$$

2. Чтобы получить последовательность длины n , **заканчивающуюся нулём**:

- последняя цифра — ноль (единственный вариант),
- предыдущая ($(n - 1)$ -я) цифра не должна быть нулём (иначе получится два нуля подряд), значит количество таких последовательностей равно числу допустимых последовательностей длины $n - 1$, которые **заканчиваются ненулём**:

$$y_n = x_{n-1}.$$

Итого имеем систему:

$$\begin{cases} x_n = (K - 1) (x_{n-1} + y_{n-1}), \\ y_n = x_{n-1}. \end{cases}$$

Подставим $y_{n-1} = x_{n-2}$ (сдвиг), получаем:

$$x_n = (K - 1) (x_{n-1} + x_{n-2}).$$

А искомое $\text{cnt}_n = x_n + y_n = x_n + x_{n-1}$.

Подставим выражение для x_n :

$$\text{cnt}_n = x_n + x_{n-1} = (K - 1)(x_{n-1} + x_{n-2}) + x_{n-1}.$$

Но заметим, что $\text{cnt}_{n-1} = x_{n-1} + x_{n-2}$. Тогда

$$x_{n-1} = \text{cnt}_{n-1} - x_{n-2},$$

и подстановка даёт упрощённую и очень удобную рекурренту напрямую для cnt_n . Проще — видеть заранее и доказать другим путём:

Лучше выделить сразу рекурренту для cnt_n . Из вышнего $y_n = x_{n-1}$ и $\text{cnt}_{n-1} = x_{n-1} + x_{n-2}$ имеем

$$x_{n-1} + x_{n-2} = \text{cnt}_{n-1}.$$

Тогда

$$x_n = (K - 1) \cdot \text{cnt}_{n-1},$$

и

$$\text{cnt}_n = x_n + x_{n-1} = (K - 1)\text{cnt}_{n-1} + x_{n-1}.$$

Но $x_{n-1} = \text{cnt}_{n-1} - x_{n-2} = \text{cnt}_{n-1} - (K - 1)\text{cnt}_{n-2}$ — это громоздко. Более компактный путь — изначально вывести стандартную рекурренту, которую используют в коде:

На самом деле из системы легко вывести (коротко):

- $x_n = (K - 1)(x_{n-1} + x_{n-2}).$
- $\text{cnt}_n = x_n + x_{n-1}.$

Сдвинем индекс для $\text{cnt}_{n-1} = x_{n-1} + x_{n-2}$. Тогда

$$x_n = (K - 1) \text{cnt}_{n-1}.$$

Подставляем в $\text{cnt}_n = x_n + x_{n-1} = (K - 1)\text{cnt}_{n-1} + x_{n-1}.$

А теперь выразим $x_{n-1} = (K - 1)\text{cnt}_{n-2}$ (по той же формуле для x_{n-1}). Тогда

$$\text{cnt}_n = (K - 1)\text{cnt}_{n-1} + (K - 1)\text{cnt}_{n-2} = (K - 1)(\text{cnt}_{n-1} + \text{cnt}_{n-2}).$$

Итого получаем очень простую и удобную рекурренту:

$\text{cnt}_1 = K - 1, \quad \text{cnt}_2 = (K - 1)K, \quad \text{cnt}_n = (K - 1)(\text{cnt}_{n-1} + \text{cnt}_{n-2}) \text{ для } n \geq 3.$

(Именно эту рекурренту и реализует ваш код.)

Доказательство корректности рекуррентии (короткая индукция)

1. База $n = 1, 2$ очевидна: одна цифра — $K - 1$ вариантов; две цифры — первая $K - 1$ вариантов, вторая любой из K (нулевая и ненулевые) — итого $(K - 1)K$ (двух нулей подряд при $n = 2$ не получится, потому что первая не ноль).

2. Пусть рекуррентия верна для всех меньших длин. Рассмотрим длину n . Разделим все допустимые n -значных числа по тому, где стоит **первое** в конце ненулевое место в последнем блоке: в сущности, комбинации, заканчивающиеся ненулём образуют $(K - 1)$ умноженное на все допустимые $(n - 1)$ -последовательности; комбинации, заканчивающиеся нулём, предшествующая цифра обязана быть ненулём и т. д. Это приводит к системе выше и даёт указанную рекуррентцию. Формально шаг индукции сводится к выводу $\text{cnt}_n = (K - 1)(\text{cnt}_{n-1} + \text{cnt}_{n-2})$, что и завершает индуктивный аргумент.

Проверка на примерах

1. $N = 2, K = 10$: $\text{cnt}_2 = (10 - 1) \cdot 10 = 90$. (Совпадает с примером.)
2. $N = 4, K = 2$: посчитаем вручную по рекуррентии:
 - $\text{cnt}_1 = 1$,
 - $\text{cnt}_2 = 1 \cdot 2 = 2$,
 - $\text{cnt}_3 = 1 \cdot (2 + 1) = 3$,
 - $\text{cnt}_4 = 1 \cdot (3 + 2) = 5$. Совпадает с примером.
3. $N = 6, K = 3$:
 - $\text{cnt}_1 = 2$
 - $\text{cnt}_2 = 2 \cdot 3 = 6$
 - $\text{cnt}_3 = 2(6 + 2) = 16$
 - $\text{cnt}_4 = 2(16 + 6) = 44$
 - $\text{cnt}_5 = 2(44 + 16) = 120$
 - $\text{cnt}_6 = 2(120 + 44) = 328$. Совпадает с примером.

О коде и замечания по реализации

Ваш код:

cpp

 Копировать код

```
int cnt[20];
...
cnt[1] = base - 1;
cnt[2] = (base - 1) * base;
for(int i = 3; i <= n; i++){
    cnt[i] = (base - 1) * (cnt[i - 1] + cnt[i - 2]);
}
cout << cnt[n];
```

точно реализует описанную рекуррентцию. Пару практических замечаний:

- В коде `int` чаще всего достаточно (по ограничениям задачи $N + K \leq 18$ числа небольшие). Но в общем случае для больших N лучше использовать 64-битный тип (`long long`), поскольку счётчики растут экспоненциально.
- Инициализация `cnt` и индексация с `1` корректна.
- Временная сложность $O(N)$, память $O(N)$ (можно хранить только два последних значения, если нужно).

Мячик на лестнице

(Время: 1 сек. Память: 16 Мб Сложность: 42%)

На вершине лесенки, содержащей N ступенек, находится мячик, который начинает прыгать по ним вниз, к основанию. Мячик может прыгнуть на следующую ступеньку, на ступеньку через одну или через две. То есть, если мячик лежит на 8-ой ступеньке, то он может переместиться на 5-ую, 6-ую или 7-ую.

Требуется написать программу, которая определит число всевозможных "маршрутов" мячика с вершины на землю.

Входные данные

Входной файл INPUT.TXT содержит число N ($0 < N \leq 70$).

Выходные данные

Выходной файл OUTPUT.TXT должен содержать искомое число.

Примеры

№	INPUT.TXT	OUTPUT.TXT
1	1	1
2	4	7

```
#include <bits/stdc++.h>

using namespace std;

typedef long long LL;

LL cnt[70];

int main()
{
    int n;
    cin >> n;
    cnt[0] = 1;
    for(int i = 1; i <= n; i++){
        for(int j = max(0, i - 3); j < i; j++){
            cnt[i] += cnt[j];
        }
    }
    cout << cnt[n];
}
```

Что считается и как моделировать задачу

- Лестница имеет N ступенек, мячик стартует на вершине (на ступеньке номер N) и должен попасть на землю (ступенька 0).
- За один прыжок он может опуститься на 1, 2 или 3 ступеньки. Значит, путь — это упорядоченная последовательность чисел из множества $\{1, 2, 3\}$, сумма которых равна N .
- Требуется количество различных таких последовательностей (маршрутов).

Это классическая задача о композициях числа N с ограничением частей (части — 1,2,3).

Обозначим через $c(n)$ — число маршрутов от ступеньки n до 0. Тогда нам нужен $c(N)$.

Рекуррентное соотношение (интуиция)

Рассмотрим последний (последний по времени) прыжок, которым мячик достигает земли из некоторого состояния. Последний прыжок может иметь длину 1, 2 или 3:

- Если последний прыжок длины 1, то перед ним мячик был на ступеньке 1 (то есть осталось решить задачу для $n - 1$), количество вариантов для этой подзадачи — $c(n - 1)$.
- Если последний прыжок длины 2, то перед ним он был на $n - 2$ — $c(n - 2)$ вариантов.
- Если последний прыжок длины 3, аналогично — $c(n - 3)$.

Все эти случаи взаимоисключающие и покрывают все возможности, поэтому

$$c(n) = c(n - 1) + c(n - 2) + c(n - 3) \quad \text{для } n \geq 1,$$

при условии корректной инициализации $c(0)$, $c(1)$, $c(2)$.

Начальные условия

Надо определить $c(0)$, $c(1)$, $c(2)$.

- $c(0) = 1$. Интерпретация: если вы уже на земле ($n=0$), то «путь» длины 0 — это одна пустая последовательность (один способ ничего не делать). Это стандартная и удобная конвенция для рекуррентных задач на композиции.
- $c(1) = c(0) = 1$. Единственный маршрут: прыжок длины 1.
- $c(2) = c(1) + c(0) = 1 + 1 = 2$. Маршруты: 1 + 1 и 2.

Поэтому сначала:

$$c(0) = 1, \quad c(1) = 1, \quad c(2) = 2,$$

и для $n \geq 3$

$$c(n) = c(n - 1) + c(n - 2) + c(n - 3).$$

Это — так называемая **трибоначчи**-рекуррентия (аналог Фибоначчи, но третьего порядка).

Проверка на примере $N = 4$:

$$c(3) = c(2) + c(1) + c(0) = 2 + 1 + 1 = 4.$$

$$c(4) = c(3) + c(2) + c(1) = 4 + 2 + 1 = 7. \text{ Совпадает с примером.}$$

Таблица первых значений (от 0 до 7):

- $c(0) = 1$
- $c(1) = 1$
- $c(2) = 2$
- $c(3) = 4$
- $c(4) = 7$
- $c(5) = 13 (=7+4+2)$
- $c(6) = 24 (=13+7+4)$
- $c(7) = 44 (=24+13+7)$

Формальное доказательство корректности (индукция)

Утверждение. Для всех $n \geq 0$ число допустимых маршрутов $c(n)$ удовлетворяет $c(n) = c(n-1) + c(n-2) + c(n-3)$ с базой $c(0) = 1$, $c(1) = 1$, $c(2) = 2$.

Доказательство по индукции.

- База: $n = 0, 1, 2$ верно по вышеописанным рассуждениям.
- Индуктивный шаг: пусть для всех $k < n$ утверждение верно. Рассмотрим все маршруты длины n . Разделим их по длине последнего прыжка — 1, 2 или 3. Количество маршрутов с последним прыжком 1 равно $c(n-1)$ (это любой маршрут для $n-1$, к которому добавляем 1), с последним прыжком 2 — $c(n-2)$, с последним 3 — $c(n-3)$. Эти множества маршрутов попарно дизъюнкты и покрывают все маршруты, значит $c(n) = c(n-1) + c(n-2) + c(n-3)$. Это завершает индукцию.

Соответствие коду

Приведённый код делает ровно это:

cpp

 Копировать код

```
cnt[0] = 1;
for(int i = 1; i <= n; i++){
    for(int j = max(0, i - 3); j < i; j++){
        cnt[i] += cnt[j];
    }
}
```

Для каждого i он суммирует $cnt[i-1] + cnt[i-2] + cnt[i-3]$ (если индекс выходит за диапазон — не учитывается, поэтому используется $\max(0, i-3)$), что эквивалентно нашей рекуррентии.

Начальное условие `cnt[0]=1` задаёт пустой путь.

Пример: $i = 4$ внутренняя сумма берёт `j=1..3`: `cnt[1]+cnt[2]+cnt[3] = 1+2+4 = 7`.

Корректность типов и границы

- Ограничение $N \leq 70$. Последовательность растёт экспоненциально (корень характеристики примерно $1.839286 \dots$), оценочно $c(n)$ около $C \cdot 1.839^n$. Для $n = 70$ значение порядка 10^{18} , т. е. **входит** в 64-битный `long long` (максимум $\approx 9.22 \cdot 10^{18}$). Поэтому выбранный тип `long long` в коде безопасен.
- Временная сложность приведённой реализации $O(N)$ (точнее $O(3N) = O(N)$), память $O(N)$. Можно ещё снизить память до $O(1)$, хранятся только три последних значения.

Оптимизация (вариант с $O(1)$ памяти)

Можно переписать, храня только `a, b, c` — значения для $i - 3, i - 2, i - 1$:

cpp

 Копировать код

```
long long a = 1; // cnt[0]
if (n == 0) { cout << 1; return 0; }
long long b = 1; // cnt[1]
if (n == 1) { cout << b; return 0; }
long long c = 2; // cnt[2]
if (n == 2) { cout << c; return 0; }
for (int i = 3; i <= n; ++i) {
    long long cur = a + b + c; // cnt[i] = cnt[i-1]+cnt[i-2]+cnt[i-3]
    a = b; b = c; c = cur;
}
cout << c;
```

Итог

- Математическая модель: $c(0) = 1, c(1) = 1, c(2) = 2, c(n) = c(n-1) + c(n-2) + c(n-3)$.
- Код, который вы привели, реализует эту рекуррентную зависимость правильно.
- Время $O(N)$, память $O(N)$ (можно уменьшить до $O(1)$), `long long` достаточно для $N \leq 70$.

Если хотите, могу:

- вывести явные значения $c(n)$ для конкретного n (например, для $n = 70$);
- доказать асимптотику (показать, как найти характерное уравнение и главную экспоненту);
- или привести графическое (деревянное) разложение всех маршрутов для небольших N (чтобы визуально убедиться). Что предпочитаете?

◆ Masala mazmuni

Bizda N pog'onali zina bor. Мячик (shar) eng yuqori pog'onada turibdi va u har safar 1, 2 yoki 3 pog'ona pastga sakrashi mumkin.

Masalan, agar u 8-pog'onada bo'lsa, 7-, 6- yoki 5-pog'onaga tushishi mumkin.

Bizdan so'ralgani — мячик pastga (0-pog'onaga) tushishining nechta turli usuli (yo'li) borligini topish.

◆ Matematik model (rekursiv bog'lanish)

Keling, $c(n)$ ni — n -pog'onadan 0-pog'onaga tushish yo'llari soni deb belgilaymiz.

Agar мячик n -pog'onada bo'lsa, u uch xil usulda sakrashi mumkin:

- 1 pog'ona pastga ($n \rightarrow n-1$)
- 2 pog'ona pastga ($n \rightarrow n-2$)
- 3 pog'ona pastga ($n \rightarrow n-3$)

Demak, barcha mumkin bo'lgan yo'llar uchta kichik masalalarning yig'indisiga teng:

$$c(n) = c(n-1) + c(n-2) + c(n-3)$$

Bu tribonachchi (Tribonacci) tipidagi ketma-ketlik.

◆ Boshlang'ich shartlar

Masalani boshlash uchun dastlabki qiymatlar kerak:

- $c(0) = 1$
— Chunki agar shar allaqachon pastda bo'lsa, "hech narsa qilmaslik" ham bitta yo'l sifatida hisoblanadi.
- $c(1) = 1$
— 1 pog'onadan pastga tushishning yagona yo'li: bitta sakrash (1).
- $c(2) = 2$
— Yo'llar:
 1. 1+1 (ikki marta sakrash)
 2. 2 (bir marta 2 pog'ona sakrash)

Shuning uchun:

$$c(0) = 1, \quad c(1) = 1, \quad c(2) = 2$$

va

$$c(n) = c(n-1) + c(n-2) + c(n-3) \quad (n \geq 3)$$

◆ Misol orqali tushuntirish

n	Hisoblash	Natija	Izoh
0	—	1	pastda bo'lish
1	$c(0)$	1	faqat 1 sakrash
2	$c(1)+c(0)$	2	(1+1) va (2)
3	$c(2)+c(1)+c(0)=2+1+1$	4	
4	$c(3)+c(2)+c(1)=4+2+1$	7	misoldagi javob
5	$7+4+2$	13	
6	$13+7+4$	24	
7	$24+13+7$	44	

◆ Induksiya orqali isbot

Da'vo:

$c(n) = c(n-1) + c(n-2) + c(n-3)$ formulasi barcha n uchun to'g'ri.

Isbot:

- **Baza:**

$n = 0, 1, 2$ uchun natijalar qo'lda hisoblab tekshirilgan.

- **Induksiya qadami:**

Faraz qilamiz, $c(k)$ formulasi $k < n$ uchun to'g'ri.

Endi n -pog'onadan tushish uchun мячик oxirgi sakrashni:

- 1 pog'ona ($\rightarrow c(n-1)$ yo'l)
- 2 pog'ona ($\rightarrow c(n-2)$ yo'l)
- 3 pog'ona ($\rightarrow c(n-3)$ yo'l)

qilib amalga oshiradi.

Bu holatlar o'zaro kesishmaydi va hammasini qamrab oladi.

Shuning uchun:

$$c(n) = c(n-1) + c(n-2) + c(n-3)$$

Demak, formula induksiya orqali isbotlandi.

◆ Dasturdagi har bir qatorning ma'nosi

cpp

Копировать код

```
cnt[0] = 1;
for (int i = 1; i <= n; i++) {
    for (int j = max(0, i - 3); j < i; j++) {
        cnt[i] += cnt[j];
    }
}
cout << cnt[n];
```

- `cnt[i]` – bu $c(i)$, ya'ni i -pog'onadan 0-pog'onagacha tushish yo'llari soni.
- Ichki siklda `cnt[i] = cnt[i-1] + cnt[i-2] + cnt[i-3]` hisoblanadi (chegarada ehtiyotlik bilan).
- `cnt[0] = 1` boshlang'ich shart sifatida berilgan.
- Oxirida `cnt[n]` – javob.

◆ Murakkablik tahlili

- **Vaqt murakkabligi:** $O(N)$
(har bir i uchun atigi 3 ta qiymat yig'iladi)
- **Xotira murakkabligi:** $O(N)$
(faqat bitta massiv kerak, xohlasa 3 ta o'zgaruvchida saqlasa ham bo'ladi)
- **Tip tanlovi:**
 $N \leq 70$ uchun `long long` yetarli, chunki qiymatlar 10^{18} dan oshmaydi.

◆ Yakuniy xulosa

Formulani umumlashtirsak:

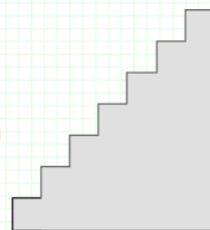
$$\begin{aligned} c(0) &= 1, \\ c(1) &= 1, \\ c(2) &= 2, \\ c(n) &= c(n-1) + c(n-2) + c(n-3) \quad (n \geq 3) \end{aligned}$$

Kod ham aynan shu formulani bajaradi va javobni aniq beradi.

Зайчик

(Время: 1 сек. Память: 16 Мб Сложность: 55%)

В нашем зоопарке появился заяц. Его поместили в клетку, и чтобы ему не было скучно, директор зоопарка распорядился поставить в его клетке лесенку. Теперь наш зайчик может прыгать по лесенке вверх, перепрыгивая через ступеньки. Лестница имеет определенное количество ступенек N . Заяц может одним прыжком преодолеть не более K ступенек. Для разнообразия зайчик пытается каждый раз найти новый путь к вершине лестницы. Директору любопытно, сколько различных способов есть у зайца добраться до вершины лестницы при заданных значениях K и N . Помогите директору написать программу, которая поможет вычислить это количество. Например, если $K=3$ и $N=4$, то существуют следующие маршруты: $1+1+1+1$, $1+1+2$, $1+2+1$, $2+1+1$, $2+2$, $1+3$, $3+1$. Т.е. при данных значениях у зайца всего 7 различных маршрутов добраться до вершины лестницы.



Входные данные

В единственной строке входного файла INPUT.TXT записаны два натуральных числа K и N ($1 \leq K \leq N \leq 300$). K - максимальное количество ступенек, которое может преодолеть заяц одним прыжком, N - общее число ступенек лестницы.

Выходные данные

В единственную строку выходного файла OUTPUT.TXT нужно вывести количество возможных вариантов различных маршрутов зайца на верхнюю ступеньку лестницы без ведущих нулей.

Примеры

№	INPUT.TXT	OUTPUT.TXT
1	1 3	1
2	2 7	21
3	3 10	274

```
k, n = map(int, input().split())
a = [1, 1]
for i in range(2, n + 1):
    a.append(sum(a[len(a) - min(k, len(a)) :]))
print(a[n])
```

1) Masala modeli (matematik talqini)

Zaychak boshidan 0-pog'onadan boshlaydi deb olamiz va n -pog'onaga chiqmoqchi. (Kodda $a[i]$ — i -pog'onaga chiqish usullari soni, shuningdek $a[0] = 1$ deb olinadi — bo'sh yo'l bitta hisoblanadi.)

Har bir sakrash 1 dan K gacha bo'lgan butun pog'ona uzunligida bo'lishi mumkin. Shunday qilib, n ni $1..K$ qismlarga bo'luvchi (tartib muhim) barcha kombinatsiyalarni hisoblash kerak — bu **composition** muammosi.

2) Rekurensiya (asosiy formula)

Agar $a[i]$ — i -pog'onaga yetib borish yo'llari soni bo'lsa, so'nggi sakrash uzunligi $1, 2, \dots, K$ dan biriga teng bo'ladi. Shunday qilib:

$$a[i] = \sum_{j=1}^{\min(K,i)} a[i-j]$$

va boshlang'ich shartlar:

$$a[0] = 1, \quad a[1] = 1.$$

($a[1] = 1$ — faqat bitta yo'l: 1 pog'ona sakrash.)

Koddagi `a` ro'yxatida `a[0]=1`, `a[1]=1` shunday o'rnatilgan va har bir qadamda oxirgi $\min(K, i)$ elementlar yig'iladi.

3) Kombinatorik ma'no

Har qanday yo'l oxirida so'nggi sakrash bor — u uzunligi j bo'lsa ($1 \leq j \leq K$), unda oldingi qism $i - j$ pog'onagacha yetib borishning $a[i - j]$ usuli bor. Barcha mumkin j larni yig'ib, umumiy sonni olamiz.

4) Induktiv isbot (to'g'riligi)

Baza: $i = 0$ uchun $a[0] = 1$ to'g'ri. $i = 1$ uchun $a[1] = 1$ to'g'ri.

Induksiya: Faraz qilaylik, formula barcha $< i$ lar uchun to'g'ri. Endi i uchun qaraymiz: oxirgi sakrash $1..K$ dan biri bo'lishi mumkin va har bir holat $a[i - j]$ ga mos keladi. Bu holatlar o'zaro kesishmaydi va hammasini jamlasak $a[i]$ hosil bo'ladi. Demak formula to'g'ri. Induksiya yakunlandi.

5) Misollar (koddagi natijalar)

1) $K = 1, N = 3$

Faqat 1 pog'onali sakrash mumkin, shuning uchun bitta yo'l: $1 + 1 + 1$.

Kod: `a = [1,1]` → `a[2]=1` → `a[3]=1` → natija `1`.

2) $K = 2, N = 7$ (masala misoli)

Bu — Fibonacci turiga yaqin hisob. Kod chiqarishi `21` (masala bergan natija).

3) $K = 3, N = 10$

Aynan keltirilgan misol: natija 274.

Agar $K = 3$ uchun butun qatorni yozsak:

$$a[0..10] = [1, 1, 2, 4, 7, 13, 24, 44, 81, 149, 274]$$

(ya'ni $a[4] = 7$ — siz bergan $N = 4, K = 3$ misoli uchun 7 yo'l).

6) Kod qanday ishlaydi, qadamma-qadam

- `a = [1,1]` — boshlang'ich: $a[0] = 1, a[1] = 1$.
- `for i in range(2, n+1):` — har bir i uchun $a[i]$ ni hisoblaymiz.
- `a.append(sum(a[len(a)-min(k, len(a)):]))` — ro'yxatning oxirgi `min(k, len(a))` elementlarini yig'adi; bu sum oxirgi K (yoki mavjud bo'lsa kamroq) qiymatlarni oladi: ya'ni formula $\sum_{j=1}^{\min(K,i)} a[i-j]$ ni amalga oshiradi.
- Oxirida `print(a[n])` chiqaradi.

7) Murakkablik va amaliy jihatlari

- Hozirgi kodda har bir i uchun sum up to K element olinadi $\rightarrow$ vaqt murakkabligi $O(N \cdot K)$.
 $N, K \leq 300$ uchun bu butunlay yetarli va juda tez.
- Python'da butun sonlar cheksiz aniqlikda saqlanadi, shuning uchun katta natija ham xavfsiz.
- Agar N juda katta bo'lganida $O(N \cdot K)$ sekin bo'lishi mumkin — shu holatda **sliding window** yordamida $O(N)$ vaqtga optimallashtirish mumkin (pastda misol):

Sliding-window optimizatsiyasi ($O(N)$):

python

 Копировать код

```
k, n = map(int, input().split())
a = [1, 1]
window = 2 # a[0]+a[1] = 1+1 = 2
for i in range(2, n + 1):
    nxt = window
    a.append(nxt)
    window += nxt
    if i - k >= 0:
        window -= a[i - k] # usulni tutib turuvchi oynadan chiqarganimiz
print(a[n])
```

Bu yerda `window` har qadam oxirgi up to K qiymatlar yig'indisini ushlab turadi, shuning uchun har qadam $O(1)$.

8) Yakuniy xulosa

- Yechim rekursiv formula: $a[i] = \sum_{j=1}^{\min(K,i)} a[i-j]$ va $a[0] = 1$, $a[1] = 1$.
- Kod shu formulani to'g'ri amalga oshiradi va misollar bilan mos keladi.
- $N, K \leq 300$ sharoitida berilgan yechim xavfsiz va tez — kerak bo'lsa uni sliding-window yordamida yanada samarali qilish mumkin.

Agar xohlasangiz, men siz uchun:

- ma'lum K, N uchun butun yo'llar daraxtini (grafik) chizib beraman (kichik N uchun, masalan $N = 4, K = 3$), yoki
- sizga barcha marshrutlarni (ro'yxatini) chizaman yoki sanab beraman. Qaysi variant kerak?